The tools and techniques of R programming
Lesson-10
DataFile Handling with R

By
Prof.Dr. A.B.Chowdhury,HOD,CA

Techno India University, West Bengal,India
Email-Id:hod.ca@technoindiaeducation.com

December 5, 2023

Introduction to Files

A **file** may be defined as an organized collection of interrelated, well-ordered, and self-contained useful facts held on a permanent storage medium. The facts may be a sequential set of instructions in a programming language held for the purpose of execution in a computer or it may be some text held for later use or it may be well-related data in the form of records about similar set of entities. In the first case, it is called a **program file** while in the latter cases, the collection is known as a **data file**.

By default, the term file refers to a data file in which each record is an ordered set of attribute values of an entity. The records may either be of fixed-length or of variable length. An **attribute** implies a characteristic of an **entity.** An **entity** is something about which we become interested and hence collect data. It is also known as a field. For example, if we consider the students of a batch, then it forms a well-related and self-contained collection of similar entities;now, for each student, we may like to collect data on his/her id number,name,phone number etc., then these will be characteristics or attributes about each of the students for which we would like to have data.

Concepts of datafiles Contd. and its handling in R

As the values on these characteristics are always stored in a specific sequence which is kept same for all the students for ease of handling, we obtain an ordered collection of attribute values for each of the students i.e. records about the set of entities(students here). If there is a definite way of keeping the records in the storage medium and getting them back in case of necessity, we say that the records have been well organized. Finally, we would expect the records of all the students of the batch together in the storage, thus making the collection a self-contained one. The storage media may be a disk or a printed paper.Earlier, files were kept on magnetic tapes also.

File Handling in R

R is very powerful in handling files of a variety of types. Most often R is used to read data from files of various formats created in some other systems for the purpose of data manipulation with a view to generate desired results. Frequently we require reading files created as text, or in Excel, SPSS or Strata or in a text(txt), csv(comma separated value) format. We shall examine in details how these can be done. But before such detailed manipulations, we would like to introduce the reader to the basic concepts of creating and reading a **CSV file** in R environment .

R Commands for Datafile handling

Let us first create a separate directory for keeping the files with which we shall be dealing. The following is the command for this purpose:
dir.create(" Name of the directory")
Let us next find out our current working directory by calling the R function:
>**getwd()**
And getting the response may be as shown below:
[1] "C:/Documents and Settings/User/My Documents"
Let us now set our working directory as follows:
setwd("./ name-of-our-directory")
and check, **getwd()**
[1] "C:/Documents and Settings/User/My Documents/TEST" , Where TEST is our given directory name.
Let us now create a csv file of our choice with the name: **myfirstfile.csv**
So, we issue the command: **file.create("myfirstfile.csv")**
If any file exists with the same name, then it is truncated ; otherwise, a file is created with the given name and the system returns TRUE.

R Commands for Datafile handling–Contd.

We can also check the existence of any file with the given name by using the logical function
file.exists("file-name") before destroying an existing file. Now, we can checking whether we can write a record in the file or not by issuing the a command like the following:
cat("Welcome to R file writing", file="file-name")
If we want to change the name of any existing file later, we may issue the following command:
file.rename("existing-name", "New-name")
The system will return TRUE on successful completion of the renaming.
If we want to read data from a .csv file, we may issue the following command with a user-given name Filedata:
filedata=read.table("csv file-name", sep=",")
and then print the data:
print(Filedata)
If we we want to remove a file, we issue the command:
file.remove("file-name")

R Commands for Datafile handling–Contd.

Now, we may like to delink the file structure also from the directory by issuing the command:

unlink("file-name")

If we require appending records of one file(say,file2.csv) at the end of some other existing file(say,file1.csv), we can do that easily by issuing the command:

file.append("file1.csv",file2.csv")

We can also copy the content of one file(say, file1.csv) into the other (say,file.csv) as follows:

file.copy("file1.csv","file.csv")

We may also extract information about a file comprising information like file size,mode, date and time of creation and that of last updation etc., we may issue the command:

file.info("file-name")

We may also list the files in a directory/folder by issuing the command:

list.files(path = ".", pattern = "*.csv")

We may also construct a path to a file as follows:

file.path("path")

We may also display the directory and filename as shown below:

dirname(file.path("filename"))

R Commands for Writing data into a Datafile

The function **write.table()** is one of the base functions in R. It is generally used to export a data frame or a matrix to a text file.

A simplified format of the function is as shown below:

write.table(x, file, append = FALSE, sep = " ", dec = ".", row.names = TRUE, col.names = TRUE)

Where,

- **x** stands for a matrix or a data frame to be written.
- **file** stands for a character string specifying the name of the target file, e.g. a **.txt**. file.
- **sep** is used to state the field separator string, e.g., sep = "\t" (for tab-separated value).
- **dec** is used to denote a string to be used as decimal separator. **Default is "."**
- **row.names** is used to state either a logical value indicating whether the row names of x are to be written along with x, or a character vector of row names to be written.
- **col.names** is used to specify either a logical value indicating whether the column names of x are to be written along with x, or a character vector of column names to be written. If col.names = NA and row.names = TRUE a blank column name is added, which is the convention used for **CSV files** to be read by spreadsheets.

For creating **csv files** with **write**, we make use of the functions **write.csv()** and **write.csv2()** with the following meanings:

- **write.csv()** uses **"."** for the decimal point and a comma **(",")** for the separator.
- **write.csv2()** uses a comma **(",")** for the decimal point and a semicolon **(";")** for the separator.

R Commands for Writing data into a Datafile

The syntax of the two forms are as stated below:

write.csv()Name of the dataset, file = "desired_file_name.csv")

write.csv2(Name of the dataset, file = "desired_file_name.csv")

Later, the data can be read together from the csv file by using a command like the following one:

mydata=read.csv(filename)

It may be noted that an extra column is added by the system with the label X. It is meant for numbering the rows in the display.

To read the datafile back into a table from the file written by using the function: **write.table()** , we use the function: **read.table()**.

Let us illustrate the concept of write.table() using the built-in R **airquality** data set to a tab-separated (sep = "\t") file called **airqualities.txt** in the current working directory and read it back in the following R Code:

Loading **airquality** data

data("airquality")

Writing **airquality** data

write.table(airquality,file="airqualities.txt",sep="\t",row.names=TRUE, col.names=NA)

To read the data back from the created file, we write:

rd=read.table("airqualities.txt"), where **rd** is a given name of our choice.

An R Script on handling .csv files

#file_creation_1 shows in steps how to accept inputs to form a data frame on two pre-decided columns/variables and then displays them back

filename=readline(prompt="Enter a .csv filename of your Choice")
file.create(filename)
cat("How many records to write in the file?")
recno=scan(n=1,quiet=TRUE)

Now for the two fields decided earlier,namely **NAME** and **AGE**, we define two vectors with length equal to the number of records to be written as shown here.

vec1=vector(mode="character",length=recno)
vec2=vector(mode="integer",length=recno)
for(i in 1:recno){ vec1[i]=readline("Enter a Name")
vec2[i]=as.integer(readline("Enter the corresponding Age"))
} **dataset=data.frame(NAME=vec1,AGE=vec2)**

Now, we create the file with data of the data frame with the following command:

write.csv(dataset,file=filename)

Now, we can read back the data from the **.csv** file into a dataframe as follows:

dtf=read.csv(filename)

N.B. To have a look at the data read into the data frame, we write:**dtf** at the R prompt.
We can save the above R script with a filename like "filescript.R".Later, we can use the following command at the R prompt: **source("filescript.R")** to load and execute the script of R commands.

A Generalised R Script for handling .csv files

The following R script enables the user to input not only the filename and number of records to be written in a file, but also the number of fields to be contained in each record,inputting the desired field names and their corresponding datatype to create a dataframe of variable dimensions that can be written into the named file.

```
{
filename=readline("Enter a filename of your Choice=>")
file.create(filename)
cat("How many records would you like to write in the named file?")
recno=scan(n=1,quiet=TRUE)
cat("How many fields/columns will there be in each record?")
fno=scan(n=1,quiet=TRUE)
df=data.frame(matrix(nrow=recno,ncol=0)) # This creates an empty dataframe marked with the number
for(i in 1:fno){ cat("For field number",i,"Enter the field name: ")
fname=readline()
print("Enter the data type of the field: ")
print(" Type 1 for numeric,2 for integer,3 for character")
print("4 for logical and 5 for date")
ft=scan(n=1,quiet=TRUE)
vec=switch(ft,vector(mode="numeric",length=recno),vector(mode="integer",length=recno),vector(mode="character",
length=recno),vector(mode="logical",length=recno),vector(mode="date",length=recno))
cat("For the field named as",fname)
cat("Enter ",recno," pieces of data for the whole column") cat("\n")
print("Type in the corresponding data for the whole column")
print("For Date data use: 'yyyy-mm-dd' format")
for(j in 1:recno){
vec[j]=switch(ft,as.numeric(readline()),as.integer(readline()),readline(),as.logical(readline()),as.Date(readline())) }
df1=data.frame(fname=vec)
colnames(df1)[1]=c(fname)
df=cbind(df,df1)}
write.csv(df,file=filename)
cat("The desired file: ",filename," has been created Successfully")
cat("We are now reading the records from the csv file into a dataframe")
data=read.csv(filename)
cat("The file has been read successfully into a data frame which is being displayed now")
data}
```

Redirecting the Console output in R

The content of the current file directory can be seen by issuing the command: **dir()**
We can redirect the console output to files of our choice.Here is a list of such files.

- To redirect console output to a **text file**, we write:**sink("myfilename.txt")**
- To send the subsequent graphical output to a **pdf file**, we may write: **pdf("mygraphs.pdf")**
- To send the subsequent graphical output to a file of **type PNG**, we may write: **png("mygrahs.png")**
- To redirect the graphical output to files of **JPEG type**, we may write: **jpeg("mygraphs.jpeg")**
- To redirect the graphical output to file of **BMP**, we may write: **bmp("mygraphs.jpeg")**
- To redirect the graphical output to a **Post-Script file**, we may write: **postscript("mygraphs.ps")**
- To come back to the screen output from redirecting the graphical output, we are to write: **dev.off()**
- To comeback to the console output mode from redirecting to the text file mode, we write: **Sink()**

Illustrative use of sink()

To redirect the console output to a file,we decide a text file name and declare it as follows at the R Command Prompt: **sink("newfile.txt")**

We next write a few **cat()** statements with some text each followed by a newline command as illustrated below:

>**cat("I am a professor of Computer Science and Engineering Applications")**

>**cat("\n")**

>**cat("I am MCA,MBA,M.Tech,PhD,FIETE,FIIE,MIE,MCSI")**

We next stop redirecting console output to the named file by issuing the following statement: **sink()**

Now,to show the content of the file, we write: **file.show("newfile.txt")**

This shows the lines written in the file: **"newfile.txt"**, one by one as displayed below:

I am a professor of Computer Science and Engineering Applications
I am MCA,MBA,M.Tech,PhD,FIETE,FIIE,MIE,MCSI

The three **cat() statements** above can also be replaced with a single statement as shown below to generate the same output:
cat("I am a professor of Computer Science and Engineering Applications","I am MCA,MBA,M.Tech,PhD,FIETE,FIIE,MIE,MCSI", file="output.txt",sep="\n",append=TRUE)

Saving and loading different R objects

The **save()** is used to write an external representation of R objects to the specified file. The objects can be read back from the file later by using the command function **load() ,or attach() (or data() function in some cases).**
The general syntax of the **save()** is as shown below:
save(..., list = character(),file = stop("'file' must be specified"),ascii = FALSE, version = NULL, envir = parent.frame(), compress = isTRUE(!ascii), compression_level, eval.promises = TRUE, precheck = TRUE)
save.image(file = ".RData", version = NULL, ascii = FALSE,compress = !ascii, safe = TRUE)
save.image() is just a short-cut for 'save my current workspace', i.e., **save(list = ls(all.names = TRUE), file = ".RData", envir = .GlobalEnv)**. It is also what happens with **q("yes")**.
Where,
... the names of the objects to be saved (as symbols or character strings).
list A character vector containing the names of objects to be saved.
file a (writable binary-mode) connection or the name of the file where the data will be saved (when tilde expansion is done). Must be a file name for **save.image or version = 1.**
ascii if TRUE, an ASCII representation of the data is written. The default value of ascii is FALSE which leads to a binary file being written. If NA and version >= 2, a different ASCII representation is used which writes double or complex numbers as binary fractions.
version the workspace format version to use. NULL specifies the current default format (3). Version 1 was the default from R 0.99.0 to R 1.3.1 and version 2 from R 1.4.0 to 3.5.0. Version 3 is supported from R 3.5.0.
envir environment to search for objects to be saved.
compress logical or character string specifying whether saving to a named file is to use compression. TRUE corresponds to gzip compression, and character strings "gzip","bzip2" or "xz" specify the type of compression. Ignored when file is a connection and for workspace format version 1.
compression_level integer: the level of compression to be used. Defaults to 6 for gzip compression and to 9 for bzip2 or xz compression.
eval.promises logical: should objects which are promises be forced before saving?
precheck logical: should the existence of the objects be checked before starting to save (and in particular before opening the file/connection)? Does not apply to version 1 saves.

safe logical. If **TRUE**, a temporary file is used for creating the saved workspace. The temporary file is renamed to file if the save succeeds. This preserves an existing workspace file if the save fails, but at the cost of using extra disk space during the save.

Description of the arguments–Contd.

The names of the objects specified either as symbols (or character strings) in ... or as a character vector in list are used to look up the objects from **environmentenvir**. By default promises are evaluated, but if **eval.promises = FALSE**,promises are saved (together with their evaluation environments). (Promises embedded in objects are always saved unevaluated.)
All R platforms use the **XDR (bigendian)** representation of C ints and doubles in binary save-d files, and these are portable across all R platforms. ASCII saves used to be useful for moving data between platforms but are now mainly of historical interest. They can be more compact than binary saves where compression is not used, but are almost always slower to both read and write: binary saves compress much better than ASCII ones. Further, decimal ASCII saves may not restore double/complex values exactly, and what value is restored may depend on the R platform.
Default values for the **.ascii, compress, safe and version** arguments can be modified with the**"save.defaults"**option (used both by save and save.image), If a "save.image.defaults" option is set it is used in preference to "save.defaults" for function save.image (which allows this to have different defaults). In addition, compression_level can be part of the "save.defaults" option.
A connection that is not already open will be opened in mode "wb". Supplying a connection which is open and not in binary mode gives an error.
Compression
Large files can be reduced considerably in size by compression. A particular 46MB R object was saved as 35MB without compression in 2 seconds, 22MB with gzipcompression in 8 secs, 19MB with bzip2 compression in 13 secs and 9.4MB with xz compression in 40 secs. The load times were 1.3, 2.8, 5.5 and 5.7 seconds respectively. These results are indicative, but the relative performances do depend on the actual file: xz compressed unusually well here.
It is possible to compress later **(with gzip, bzip2 or xz)** a file saved with **compress = FALSE**: the effect is the same as saving with compression. Also, a saved file can be uncompressed and re-compressed under a different compression scheme .
Example of Saving:
Let us consider the following statements:
>**m=matrix(101:120,nrow=5)** # This is to create a matrix
>**df=as.data.frame(m)** # This is to convert the matrix into a dataframe
>**save(df,file='matrix.data')** # This is to save the dataframe into a file with a desired name.
An important point to be kept in mind in specifying the path of a file is that in R, file paths are always specified with forward slashes ("/"), So, to save this file to the directory C:*Documents**myself**MyDocuments*\,
we would use the following command:
save(df,file='c:/Documents/myself/Mydocuments/matrix.dat')
To read back this file, we refer to it as shown below:
dataset=read.table('c://Documents//myself//Mydocuments//matrix.dat')

Examples of statements for saving files

1.Let us suppose that we want to save our data frame social_status under the current working directory, so, we issue the command:
save(social_status,file="~/status.RDATA") to save it
with the name **"status.RDATA"**
2.Let us create two vectors and one list and save them together in a single file as shown below:

```
>a=c(1,2,3,4)
>b=c(5,6,7,8)
>c=list(A=c(9,10,11),B=c("RAM","SHYAM","JADU"))
>save(a,b,c,file="abc.Rdata")
>save.image() # saving the file "abc.Rdata" in the current directory
>unlink(".Rdata")
```

To reload the datasets written with the function **save()**, we use the **load()** function which has the following syntax:

load(file,envir=parent.frame() ,verbose=FALSE)

Where,

file refers to a (readable binary-mode) connection or a character string giving the name of the file to load (when tilde expansion is done).
envir refers to the environment where the data should be loaded.
verbose is logical It implies whether the item names are required to be printed during loading
More Details
The function can load R objects saved in the current or any earlier format. It can also read a compressed file directly from a file or from a suitable connection including a call to a **url**.

Syntax explanation of load() contd.

An **unopen connection** will be opened in the "rb" mode and then closed after use.

Only the R objects saved in the current format (used since R1.4.0) can be read from a connection. If no input is available on a connection, the system will generate a warning; but any input which is not in the current format will result in an error. The **verbose argument** is basically intended for debugging purpose. If it is set to TRUE, then as objects from the file are loaded, their names will be displayed on the console.

Objects can be saved with references to **namespaces**, usually as part of the environment of a function or formula. Such objects can be loaded even if the **namespace** is not available: it is replaced by a reference to the global environment with a warning.

load(<file >) replaces all existing objects with the same names in the current environment (typically the current workspace,**.GlobalEnv)** and hence potentially overwrites important data. It is considerably safer to **use envir =** to load into a different environment, or to **attach(file)** which **load()s** into a new entry in the search path.

Importing datafiles from external sources

R is equipped with an extremely beautiful capability of reading data from many different software and programming environments readily even if the file is mentioned through an url. It is fairly simple to read data from many important sources like **STATA, EXCEL, SPSS, SAS, SYSTAT and Database systems,** to name a few, for the purpose of data analysis using the powerful functions of R.

The **read.table()**

read.table() is a very flexible family of functions provided by R for importing files with many different properties. The function reads a file in table format and creates a data frame from it, with cases corresponding to lines and variables to fields in the file. The syntax of the function is as stated below:

read.table(file, header = FALSE, sep = "", quote = "\", dec = ".", numerals = c("allow.loss", "warn.loss", "no.loss"),
row.names, col.names, as.is = !stringsAsFactors,
na.strings = "NA", colClasses = NA, nrows = -1,
skip = 0, check.names = TRUE, fill = !blank.lines.skip, strip.white = FALSE, blank.lines.skip = TRUE,
comment.char = "#", allowEscapes = FALSE, flush = FALSE, stringsAsFactors = default.stringsAsFactors(), fileEncoding = "", encoding = "unknown", text, skipNul = FALSE)

More Read commands in R

read.csv(file, header = TRUE, sep = ",", quote = "$\backslash$", dec = ".", $fill$ = $TRUE$, $comment.char$ = "", ...)

read.csv2(file, header = TRUE, sep = ";", quote = "$\backslash$", dec = ",", $fill$ = $TRUE$, $comment.char$ = "", ...)

read.delim(file, header = TRUE, sep = "$\backslash t$", $quote$ = "$\backslash$", dec = ".", $fill$ = $TRUE$, $comment.char$ = "", ...)

read.delim2(file, header = TRUE, sep = "$\backslash t$", $quote$ = "$\backslash$", **dec = ",", fill = TRUE, comment.char = "", ...)**

Where, the arguments possess the meanings described below:

file the name of the file from which the data are to be read. Each row of the table appears as one line of the file. If it does not contain an absolute path, the file name is relative to the current working directory. Tilde-expansion is performed where supported. This can be a compressed file.

Alternatively,file can be a readable text-mode connection (which will be opened for reading if necessary, and if so closed (and hence destroyed) at the end of the function call). (If **stdin()** is used, the prompts for lines may be somewhat confusing. The input is required to be terminated with a blank line or an **EOF** signal,**Ctrl-D on Unix** and **Ctrl-Z on Windows**. Any **pushback on stdin()** will be cleared before return.)

file can also be a complete URL.

header is a logical value indicating whether the file contains the names of the variables as its first line. If missing, the value is determined from the file **format:header.** It is set to TRUE if and only if the first row contains one fewer field than the number of columns.

sep implies the field separator character. Values on each line of the file are separated by this character. If **sep = ""** (the default for read table) , the separator is **'white space'**, that is one or more spaces, tabs, newlines or carriage returns.

quote implies the set of quoting characters. To disable quoting altogether, we use **quote = ""**. Quoting is only considered for columns read as character, which is all of them unless **colClasses** is specified.

dec implies the character used in the file for decimal points.

numerals implies string indicating how to convert numbers whose conversion to double precision would lose accuracy,

row.names indicates a vector of row names. This can be a vector giving the actual row names, or a single number giving the column of the table which contains the row names, or character string giving the name of the table column containing the row names.

Meaning of the arguments in the read() contd.

If there is a header and the first row contains one fewer field than the number of columns, the first column in the input is used for the row names. Otherwise if row.names is missing, the rows are numbered.
Using **row.names = NULL** forces row numbering. Missing or NULL row.names generate row names that are considered to be 'automatic' (and not preserved by as.matrix). **col.names** indicates a vector of optional names for the variables. The default is to use "V" followed by the column number.
as.is is the default behavior of read.table is to convert character variables (which are not converted to logical, numeric or complex) to factors. The variable as is controls the conversion of columns not otherwise specified by **colClasses** . Its value is either a vector of logicals (values are recycled if necessary), or a vector of numeric or character indices which specify which columns should not be converted to factors.
To suppress all conversions including those of numeric columns, we set **colClasses = "character"**.
It may be noted that **as.is** is specified per column (not per variable) and so includes the column of row names (if any) and any columns to be skipped.
na.strings a character vector of strings which are to be interpreted as NA values. Blank fields are also considered to be missing values in logical, integer, numeric and complex fields. It should be noted that the test happens after white space is stripped from the input, so, na.strings values may need their own white space stripped in advance.
colClasses character. A vector of classes to be assumed for the columns. If unnamed, recycled as necessary. If named, names are matched with unspecified values being taken to be NA.
Possible values are NA (the default, when type convert is used),"NULL"(when the column is skipped), one of the atomic vector classes (logical, integer, numeric, complex, character, raw), or "factor", "Date" or "POSIXct". Otherwise there needs to be an as method (from package methods) for conversion from "character" to the specified formal class.
It should be noted that **colClasses** is specified per column (not per variable) and so includes the column of row names (if any).
nrows integer: the maximum number of rows to read in. Negative and other invalid values are ignored.
skip integer: the number of lines of the data file to skip before beginning to read data.
check.names logical. If TRUE then the names of the variables in the data frame are checked to ensure that they are syntactically valid variable names. If necessary they are adjusted (by make.names) so that they are, and also to ensure that there are no duplicates.
fill logical. If TRUE then in case the rows have unequal length, blank fields are implicitly added.
strip.white logical. Used only when sep has been specified, and allows the stripping of leading and trailing white space from unquoted character fields (numeric fields are always stripped).
blank.lines.skip logical: if TRUE blank lines in the input are ignored.
comment.char character: a character vector of length one containing a single character or an empty string. We use "" to turn off the interpretation of comments altogether.
allowEscapes logical. Should C-style escapes such as be processed or read verbatim (the default)? Note that if not within quotes these could be interpreted as a delimiter (but not as a comment character).

Further details about read()

flush logical: if TRUE, scan will flush to the end of the line after reading the last of the fields requested. This allows putting comments after the last field.
stringsAsFactors logical: should character vectors be converted to factors? Note that this is overridden by as.is and colClasses, both of which allow finer control.
fileEncoding character string: if non-empty declares the encoding used on a file (not a connection) so the character data can be re-encoded.
encoding encoding to be assumed for input strings. It is used to mark character strings as known to be in Latin-1 or UTF-8 .it is not used to re-encode the input, but allows R to handle encoded strings in their native encoding (if one of those two).
text character string: if file is not supplied and this is, then data are read from the value of text via a text connection. It may be noted that a literal string can be used to include (small) data sets within R code.
skipNul logical: should nuls be skipped?
... Further arguments to be passed to read.table.
This function is the principal means of reading tabular data into R.
Unless **colClasses** is specified, all columns are read as character columns and then converted using type convert to logical, integer, numeric, complex or (depending on **as.is**) factor as appropriate. Quotes are (by default) interpreted in all fields, so a column of values like "42" will result in an integer column.
A field or line is 'blank' if it contains nothing (except white space if no separator is specified) before a comment character or the end of the field or line. If **row.names** is not specified and the header line has one less entry than the number of columns, the first column is taken to be the row names. This allows data frames to be read in from the format in which they are printed. If row.names is specified and does not refer to the first column, that column is discarded from such files. The number of data columns is determined by looking at the first five lines of input (or the whole input if it has less than five lines), or from the length of **col.names** if it is specified and is longer.
This could conceivably be wrong if fill or blank.lines.skip are true, so specify col.names,if necessary read.csv and read.csv2 are identical to read.table except for the defaults. They are intended for reading 'comma separated value' files ('.csv') or (read.csv2) the variant used in countries that use a comma as decimal point and a semicolon as field separator. Similarly, **read.delim** and **read.delim2** are for reading delimited files, defaulting to the TAB character for the delimiter. It may be noticed that header = TRUE and fill = TRUE in these variants, and that the comment character is disabled.
The rest of the line after a comment character is skipped; quotes are not processed in comments. Complete comment lines are allowed provided blank.lines.skip = TRUE; however, comment lines prior to the header must have the comment character in the first non-blank column.
Quoted fields with embedded newlines are supported except after a comment character. Embedded nulls are unsupported: skipping them (with **skipNul = TRUE**) may work.

Further details about read()– Contd.

Empty input is an error unless **col.names** is specified, when a 0-row data frame is returned: similarly giving just a header line if **header = TRUE** results in a 0-row data frame. Note that in either case the columns will be logical unless **colClasses** was supplied. Character strings in the result (including factor levels) will have a declared encoding if encoding is "latin1" or "UTF-8". **Memory usage**

These functions can use a surprising amount of memory when reading large files.

Less memory will be used if **colClasses** is specified as one of the six **atomic vector** classes. This can be particularly so when reading a column that takes many distinct numeric values, as storing each distinct value as a character string can take up to 14 times as much memory as storing it as an integer.

Using **nrows**, even as a mild over-estimate, will help memory usage.

Using **comment.char = ""** will be appreciably faster than the **read.table** default.

read.table is not the right tool for reading large matrices, especially those with many columns: it is designed to read data frames which may have columns of very different classes. In such cases ,we should use scan instead for matrices.

Note

The columns referred to in **as.is** and **colClasses** include the column of row names (if any).

There are two approaches for reading input that is not in the local encoding. If the input is known to be UTF-8 or Latin1, we use the encoding argument to declare that. If the input is in some other encoding, then it may be translated on input. The **fileEncoding argument** achieves this by setting up a connection to do the re-encoding into the current locale. It may be noted that on Windows or other systems not running in a UTF-8 locale, this may not be possible.

The following example shows how to read data from the link to a datafile:

>**example = read.table("https://stats.idre.ucla.edu/stat/data/test.txt", header = TRUE)**

Let us now see what has been read in **example**.

>**example**

This shows the table below with all the headers:

	prgtype	gender	id	ses	schtyp	level
1	general	0	70	4	1	1
2	vocati	1	121	4	2	1
3	general	0	86	4	3	1
4	vocati	0	141	4	3	1
5	academic	0	172	4	2	1
6	academic	0	113	4	2	1
7	general	0	50	3	2	1
8	academic	0	11	1	2	1

Examples of read in different forms

1.**Reading from a comma-delimited text file:**
Let us consider a comma-delimited text file in the directory "C:/Documents and Settings/User/My Documents/TEST" with the name myfirstfile.csv and the current woking directory: is "C:/Documents and Settings/User/My Documents" To read the file, we may issue the following command:
mytext=read.table("∼/TEST/myfirstfile.csv",header=TRUE, sep=",",row.names="id")
2.**To read from a EXCEL file,**
One of the best ways to read an Excel file is to export it to a comma delimited file and import it using the method above. Alternatively, we can use the xlsx package to access Excel files. This is done as follows:
We install the xlsx package first by issuing the command:
install.packages("xlsx")
We may next verify the installation as shown under:
any(grep1("xlsx",installed.packages()))
The first row should contain variable/column names.

Examples of read in different forms–Contd.

Let us read in the first worksheet from the workbook: **myworks.xlsx** in which the first row contains variable names,

library(xlsx)

library(gdata)

myworks = read.xlsx("c:/ myworks.xlsx ,1)

Let us illustrate an alternative way to read in the worksheet named myworksheet:

myworks= read.xlsx("c:/ myworksheet.xlsx", sheetName = "mysheet")

Sometimes, the installation of the xlsx package does not work resulting into an error message as illustrated below:

package '. . . . .' is not available (for R version 3.5.0)

So, we mention a third alternative below:

We can import the data interactively from an excel sheet into R by using the Clipboard to copy and paste data.

To import data from the Clipboard, we use the readClipboard() function. For example, let us select cells B2:B10 in a spreadsheet, then we press Ctrl+C to copy those cells to the Clipboard, and then we use the following R code:

>**myexceldata=readClipboard()**

However, this approach works very well for vector data (in other words, a single column or row of data). But things get just a little bit more complicated when we want to import tabular data to R.

Examples of read in different forms–Contd.

To copy and paste tabular data from a spreadsheet, We may make use of the very powerful **read.table() function**.We can customize the behavior of the **read.table()** by changing its many arguments. We have to pay special attention to the following arguments:

- **file:**The name of the file to import. To use the Clipboard, we specify, file = "clipboard".
- **sep:** The separator between data elements. In the case of Microsoft Excel spreadsheet data copied from the Clipboard, the separator is a tab, indicated by "\t". So,we specify, **sep="\t"**.
- **header:** This argument indicates whether the Clipboard data includes a header in the first row (that is, column names). Whether we specify **TRUE or FALSE** depends on the range of data that we would copy.
- **stringsAsFactors**: If **TRUE**, this argument converts strings to factors. It is **FALSE** by default.

Hence, we should write our command as follows:

Myexceldata=read.table(file="clipboard",sep="\t", header=TRUE)

The read.fwf()

Sometimes, it is required to read a **fixed width format text file**. For this purpose, we may use the **read.fwf() function** with the following format:

read.fwf(file, widths, header = FALSE, sep = "\t",skip = 0, row.names, col.names, n = -1,
buffersize = 2000, fileEncoding = "", ...)

Where the arguments are explained below:

File points out the name of the file from which the data are to be read.Alternatively,file can be a connection, which will be opened if necessary, and if so closed at the end of the function call.
Widths is an integer vector, giving the widths of the fixed-width fields (of one line), or list of integer vectors giving widths for multiline records.
Header is a logical value indicating whether the file contains the names of the variables as its first line. If present, the names must be delimited by sep.
Sep is a character used to delimit variables in the header; the separator used internally; should be a character that does not occur in the file (except in the header).
Skip implies the number of initial lines to skip.
row.names is same as explained in **read.table()**
col.names is same as explained in **read.table()**
N implies the maximum number of records (lines) to be read, defaulting to no limit.
buffersize implies the maximum number of lines to read at one time
fileEncoding is a character string: if non-empty declares the encoding used on a file (not a connection) so the character data can be re-encoded.
Further Details
Multiline records are concatenated to a single line before processing. Fields that are of zero-width or are wholly beyond the end of the line in file are replaced by NA. Negative-width fields are used to indicate columns to be skipped, e.g., -5 to skip 5 columns. These fields are not seen by read.table and so should not be included in a col.names or colClasses argument (nor in the header line, if present).
Reducing the buffersize argument may reduce memory use when reading large files with long lines. Increasing buffersize may result in faster processing when enough memory is available.

The scan() function for reading from files

The **scan() function** can be used with complex file formats It can read data into a vector or list from the console or file. The general format of its syntax is as shown under:

scan(file = "", what = double(), nmax = -1, n = -1, sep = "",
quote = if(identical(sep, "")) "" else "\", dec = ".",
skip = 0, nlines = 0, na.strings = "NA", flush = FALSE, fill = FALSE, strip.white = FALSE,
quiet = FALSE, blank.lines.skip = TRUE, multi.line = TRUE, comment.char = "", allowEscapes = FALSE,
fileEncoding = "", encoding = "unknown", text, skipNul =FALSE)

Where the arguments imply meanings as explained below:

File is the name of a file to read data values from. If the specified file is "", then input is taken from the keyboard (or whatever stdin() reads if input is redirected or R is embedded). In this case input can be terminated by a blank line or an EOF signal,Ctrl-D on Unix and Ctrl-Z on Windows. Otherwise, the file name is interpreted relative to the current working directory (given by getwd()), unless it specifies an absolute path.

The file can also be a complete URL. To read a data file not in the current encoding (for example a Latin-1 file in a UTF-8 locale or conversely) a fileconnection setting is its encoding argument (or scan's fileEncoding argument) are to be used.

what contains the type of data to be read.Here, **'type'** is used in the sense of typeof

sep is to contain a separator.By default, scan expects to read 'white-space' delimited input fields. Alternatively,sep can be used to specify a character which delimits fields. A field is always delimited by an end-of-line marker unless it is quoted. If specified this should be the empty character string (the default) or NULL or a character string containing just one single-byte character.

quote contains the set of quoting characters as a single character string or NULL. In a multibyte locale the quoting characters must be ASCII (single-byte). dec is meant for decimal point character. This should be a character string containing just one single-byte character. (NULL and a zero-length character vector are also accepted, and taken as the default.)

skip includes the number of lines of the input file to skip before beginning to read data values.

nlines if positive, the maximum number of lines of data to be read.

Explanation of the arguments in scan()

na.strings is a character vector. Elements of this vector are to be interpreted as missing (NA) values. Blank fields are also considered to be missing values in logical, integer, numeric and complex fields. It should be noted that the test is performed after white space is stripped from the input, so **na.strings** values may need their own white space stripped in advance.
flush is logical: if TRUE, scan will flush to the end of the line after reading the last of the fields requested. This allows putting comments after the last field, but precludes putting more that one record on a line.
fill is logical: if **TRUE**, scan will implicitly add empty fields to any lines with fewer fields than implied by what.
strip.white is a vector of logical value(s) corresponding to items in the what argument. It is used only when **sep** has been specified, and allows the stripping of leading and trailing **'white space'** from character fields (numeric fields are always stripped). It should be noted that white space inside quoted strings is not stripped. If strip.white is of length 1, it applies to all fields; otherwise, if **strip.white[i] is TRUE and the i-th field is of mode character** (because what[i] is) then the leading and trailing unquoted white space from field i is stripped.
quiet is logical: if FALSE (default), scan() will print a line, saying how many items have been read.
blank.lines.skip is logical: if TRUE, blank lines in the input are ignored, except when counting skip and nlines.
multi.line is logical. Only used if what is a list. If FALSE, all of a record must appear on one line (but more than one record can appear on a single line). It may be noted that using fill = TRUE implies that a record will be terminated at the end of a line.
comment.char is a character vector of length one containing a single character or an empty string. We use "" to turn off the interpretation of comments altogether (the default).
allowescapes is logical. Should C-style escapes such as be processed (the default) or read verbatim? Note that if not within quotes these could be interpreted as a delimiter (but not as a comment character). The escapes which are interpreted are the control characters \a, \b, \f, \n, \r, \t, \v and octal and hexadecimal representations like 40 and x2A. Any other escaped character is treated as itself, including backslash. Note that Unicode escapes (starting ˇor :) are never processed.
fileEncoding is a character string: if non-empty declares the encoding used on a file (not a connection nor the keyboard) so the character data can be re-encoded. encoding implies encoding to be assumed for input strings. If the value is "latin1" or "UTF-8" it is used to mark character strings as known to be in Latin-1 or UTF-8: it is not used to re-encode the input. **text is a character string:** if file is not supplied and this is, then data are read from the value of text via a text connection. **skipNul** is logical meaning, it implies whether the nulls should be skipped when reading character fields.

Further Details

The value of what can be a list of types, in which case scan returns a list of vectors with the types given by the types of the elements in what. This provides a way of reading columnar data. If any of the types is NULL, the corresponding field is skipped (but a NULL component appears in the result). The type of what or its components can be one of the six atomic vector types or NULL . 'White space' is defined for the purposes of this function as one or more contiguous characters from the set space, horizontal tab, carriage return and line feed. It does not include form feed nor vertical tab, but in Latin-1 and Windows 8-bit locales (but not UTF-8) 'space' includes the non-breaking space "0".

Further Details on the arguments of scan()

Empty numeric fields are always regarded as missing values. Empty character fields are scanned as empty character vectors, unless **na.strings contains ""** when they are regarded as missing values.
The allowed input for a numeric field is optional whitespace followed either NA or an optional sign followed by a decimal or hexadecimal constant ,or NaN, Inf or infinity (ignoring case). Out-of-range values are recorded as Inf, -Inf or 0.
For an integer field the allowed input is optional whitespace, followed by either NA or an optional sign and one or more digits (0-9): all out-of-range values are converted to NA_integer_.
If sep is the default (""), the character in a quoted string escapes the following character, so quotes may be included in the string by escaping them. If sep is non-default, the fields may be quoted in the style of .csv files where separators inside quotes (" or "") are ignored and quotes may be put inside strings by doubling them. However, if sep = "" , it is assumed by default that one wants to read entire lines verbatim.
Quoting is only interpreted in character fields and in NULL fields (which might be skipping character fields).
It may be noted that since sep is a separator and not a terminator, reading a file by scan("foo", sep = "", blank.lines.skip = FALSE) will give an empty final line if the file ends in a linefeed and not if it does not.
If **comment.char** occurs (except inside a quoted character field), it signals that the rest of the line should be regarded as a comment and be discarded. Lines beginning with a comment character (possibly after white space with the default separator) are treated as blank lines.
There is a line-length limit of 4095 bytes when reading from the console.
There is a check for a user interrupt every 1000 lines if what is a list, otherwise every 10000 items.
If file is a character string and fileEncoding is non-default, or if it is a not-already-open connection with a non-default encoding argument, the text is converted to UTF-8 and declared as such (and the encoding argument to scan is ignored).
Embedded nuls in the input stream will terminate the field currently being read, with a warning once per call to scan. The setting **skipNul = TRUE** causes them to be ignored.
if what is a list, a list of the same length and same names (as any) as what.Otherwise, a vector of the type of what.Character strings in the result will have a declared encoding if encoding is "latin1" or "UTF-8".

Examples of different forms of scan()

1.**scan("filename")**

It reads and shows all the records of the named file.However, if the file contains any character data then we need to add the **what option** as shown below:

scan("filename",what="")

Use of scan() function

The preceding command reads all the records from the file together as a string.Here scan assumes that the items of the vector are separated by whitespaces including blanks,carriage returns and line feeds(). However, we may read one line at a time also as a string by using the sep option as follows:

scan("filename",what="",sep="\n")

The **scan()** can also be used to read from the keyboard by specifying an empty string for the filename as shown below by pressing **ENTER** after entering a desired number of values:

```
>f=scan("")
1: 1234
2: 123
3: 12
4: 1
5: # We just press ENTER without typing in any value
Read 4 items
>f
[1] 1234 123 12 1
```

Here, we signal the end of input with an empty line. It may be noted that the system prompts here with the index of the next input value. If the values are separated by spaces, then, they are counted as different values.

Use of scan() function–Contd.

The **scan()** is typically applied to text files.So, we can store whole of a dataframe as illustrated below:

dataset=data.frame(NAME=c("Rashid","Ramen","Peter","Diana"), AGE=c(23,22,20,19))

Here, our example dataset consists of two columns comprising character and numeric data respectively.

If we print the dataframe, then we get the following tabular set of data.

>**dataset**

	NAME	AGE
1	Rashid	23
2	Ramen	22
3	Peter	20
4	Diana	19

Now, if we write this dataframe as a text file in our system,we would like to omit the default row names. This can be done with the following command:

write.table(dataset,# Writing the dataset as text file to the current directory

file = "mydata.txt",row.names = FALSE)

mydata=scan("mydata.txt", what = "character")

The execution of this command is ended with the following message:

Read 10 items

Use of scan() function–Contd.

Let us now print the content of mydata as follows.

> **mydata**

[1] "NAME" "AGE" "Rashid" "23" "Ramen" "22" "Peter" "20"
[9] "Diana" "19"

So, it is observed that the column names of the dataframe are also stored as data items for the file. In fact, it is stored as the first record in the file.

If we don't want this record printed, we need to skip 1 record. This can be done as shown below:

mydata2=scan("myfile.txt",what="character",skip=1)

The execution of this command shows the message: **Read 8 items**

Let us now display the items read.

>**mydata2**

[1] "Rashid" "23" "Ramen" "22" "Peter" "20" "Diana" "19"

We may also read the data into a list and display them as elements of a list as shown below:

>**mydata=scan("myfile.txt",what=list("",""))**

Read 5 records *is the message displayed in this case.*

>**mydata**

[[1]]
[1] "NAME" "Rashid" "Ramen" "Peter" "Diana"

Use of scan() function–Contd.

[[2]]
[1] "AGE" "23" "22" "20" "19"
2.**x=scan("mymatrix.csv",what="character",skip=1,quiet=TRUE)**
Here, a text file(.csv) is taken as input and a vector of strings is generated as output.
3.**scan("mine.data", skip = 1, nlines = 1)**
Here,only one line is read from the file after skipping the first record.
We write the following R script for writing records in a chosen file and then read it back by using the **scan()** function.

```
filename=readline("Enter a filename to write in=>")
repeat{
record=readline("Enter a line of text to write in the file–Press only ENTER
to stop writing=>")
if(nchar(record)==0){
break
}
write(record,filename,append=TRUE)
} thedata=scan(filename,what='character')
print(thedata)
```

Illustrative Use of scan()–Contd.

Inputs are given as illustrated below when the script is run:
Enter a line of text to write in the file–Press only ENTER to stop writing=>Ram is a boy
Enter a line of text to write in the file–Press only ENTER to stop writing=>Rani is a girl
Enter a line of text to write in the file–Press only ENTER to stop writing=>
Now, we use scan() to read the file and then to display the data written there. The response shown by R system is shown below the line of execution of **scan()** and **print()**:

>**thedata=scan(filename,what='character')**
Read 8 items
>**print(thedata)**
[1] "Ram" "is" "a" "boy" "Rani" "is" "a" "girl"

The readLines() function

The preceding functions extract pieces of the input data into separate variables, and will always be the main tool for creating vectors, matrices, and data frames. But if we are just processing text, say, from a web page or from the output of some other computer program, the data won't be arranged in the way that read table need. In cases like this, we simply want to read the text into R in such a way that we can utilize it according to our needs. The R function that does this is the **readLines()**. The **readLines() function accepts a URL or file as its first argument, and returns a vector with as many elements as there are lines from the input source. Unlike scan() or read.table(),readLines() simply extracts the text from its input source and returns each line as a character string. So whenever we encounter with "unorganized" data,readLines() is the function that will read it into R so that we can manipulate it further.**

The syntax of readLines() is as shown below:

readLines(con = stdin(), n = -1L, ok = TRUE, warn = TRUE,encoding = "unknown", skipNul = FALSE)

Where the arguments are as explained below:

Explanation of the arguments in readLines()

con indicates a connection object or a character string.
n is an integer implying the (maximal) number of lines to be read. Negative values indicate that one should read up to the end of input on the connection.
ok is logical.It means whether it is OK to reach the end of the connection before n>0 lines are read. If not, an error will be generated.
warn is logical. It warns if a text file is missing a final EOL or if there are embedded nuls in the file.
encoding It is assumed for input strings. It is used to mark character strings as known to be in Latin-1 or UTF-8: it is not used to re-encode the input. To do the latter, we specify the encoding as part of the connection con or via options(encoding=).
skipNul is logical: specifies,whether nuls should be skipped or not
If the **con** is a character string, the function calls file to obtain a file connection which is opened for the duration of the function call. This can be a compressed file.
If the connection is open it is read from its current position. If it is not open, it is opened in "rt" mode for the duration of the call and then closed (but not destroyed; we must call close to do that).
If the final line is incomplete (no final EOL marker) the behaviour depends on whether the connection is blocking or not. For a non-blocking text-mode connection the incomplete line is pushed back, silently. For all other connections the line will be accepted, with a warning.Whatever mode the connection is opened in, any of LF, CRLF or CR will be accepted as the EOL marker for a line.
Embedded nuls in the input stream will terminate the line currently being read, with a warning (unless skipNul = TRUE or warn = FALSE). If con is a not-already-open connection with a non-default encoding argument, the text is converted to UTF-8 and declared as such (and the encoding argument to readLines is ignored). Value is to contain a character vector of length the number of lines read.The elements of the result have a declared encoding if encoding is "latin1" or "UTF-8"

Examples of applications of readLines()

1. **path ="/path/to/the/filename"**

```
print(path)
conn =file(path,open="r")
lines = readLines(conn)
for (i in 1:length(lines)){
print(lines[i])
}
close(conn)
```

The code above reads a file line by line and closes it finally.

The writeLines() function

This function is used for writing text lines to a connection.The syntax of the function is:

writeLines(text, con = stdout(), sep = "\n", useBytes = FALSE)

Where,

text implies a character vector
con is used to set up connection object or a character string.
sep character string. A string to be written to the connection after each line of text.
useBytes logical.
If the **con** is a character string, the function calls file to obtain a file connection which is opened for the duration of the function call.
If the connection is open it is written from, its current position. If it is not open, it is opened for the duration of the call in **"wt" mode** and then closed again.
Normally **writeLines is used with a text-mode connection**,**and the default separator is converted to the normal separator for that platform (LF on Unix/Linux, CRLF on Windows).** For more control, a binary connection is to be opened and the precise value to be written to the file is specified in **sep**.For even more control, **writeChar** is used on a binary connection.
useBytes is for expert use. Normally (when false) character strings with marked encodings are converted to the current encoding before being passed to the connection (which might do further re-encoding). **useBytes = TRUE** suppresses the re-encoding of marked strings so they are passed byte-by-byte to the connection: this can be useful when strings have already been re-encoded by e.g.iconv. (It is invoked automatically for strings with marked encoding **"bytes"**.)

Example of an application with writeLines()

>conn=file("Mytext.txt","w")
>writeLines("I am a Professor",conn)
>writeLines("I teach Computer Science and Engineering Applications",conn)

· · · Continued. . .

Example of an application with writeLines()

```
>writeLines(c("I am MBA,MCA and M.Tech","I have also been awarded PhD in Computer Science and Engineering"," I am also a Chartered Engineer,FIETE,FIIE,MIE,MCSI"),conn)
>close(conn)
>con=file(".\\Mytext.txt","r")
>lines_In_The_File=readLines(con)
>n=length(lines_In_The_File)
>for(i in 1:n){ print(lines_In_The_File[i])}
```

The following output is displayed:

```
[1] "I am a Professor"
[1] "I teach Computer Science and Engineering Applications
[1] "I am MBA,MCA and M.Tech"
 [1] " I have also been awarded PhD in Computer Science and Engineering"
[1] "I am also a Chartered Engineer,FIETE,FIIE,MIE,MCSI"
```

Examples of applications of readLines()–Contd.

2.The following program takes data from a text file written from a dataframe data,splits the data to enable procuring of the desired components, performs processing on some of the split data, creates a new form of an output record, writes the records thus generated one by one into a specified target file,closes it and then reopens it for reading all the records one by one,displays each one until the end of the file.

```
{
infilename=readline("Enter a filename to read From=>")
con =file(infilename, 'r')
outfilename=readline("Enter the filename where you want to write your records line by line=>")
outfile =file(outfilename, 'w')
data = readLines(con, n=10)
  for (i in 2:length(data)){
    rec=unlist(strsplit(data[i], split = ",")) ## split the string at comma positions
bas=rec[3]
 gp=as.integer(bas)*1.98
outrec=paste(rec[2],gp,sep=",")
cat("The salary record: ",outrec," is being written into the target file ")
  writeLines(outrec, con=outfile) }
close(outfile)
outfile=readline("Enter a filename to read From=>")
outfl=file(outfile,'r')
outrecs=readLines(outfl,n=10)
for(i in 1:length(outrecs)){
salrec=unlist(strsplit(outrecs[i],split=","))
empid=salrec[1]
gsal=salrec[2]
cat("Salary for employee with id: ",empid, "is; ",as.integer(gsal),"")}
cat("\n\n")
cat("All the records of ",outfile,"have been processed")
print(" Thank you for using this program")
close(outfl)}
```

In the above example, first 10 lines are transferred to the file buffer at a time so that processing can be done speedily with large input files.

Exporting data to text files

Writing Data to a File: the write() function

The data (usually a matrix) x are written to a named file. If x is a two-dimensional matrix we need to transpose it to get the columns in file the same as those in the internal representation. The syntax of the function is:

write(x, file = "data", ncolumns = if(is.character(x)) 1 else 5,append = FALSE, sep = " ")

Where arguments are as described below:

x specifies the data to be written out, usually an atomic vector.
file implies a connection, or a character string naming the file to write to. If "", print to the standard output connection.
ncolumns points to the number of columns to write the data in.
append: if **TRUE** the data x are appended to the connection.
sep is a string used to separate columns. Using sep = "\t" gives tab delimited output; def
Let us have a look at some examples given below:

m= matrix(1:6, ncol = 3) # This is to create a 2 by 3 matrix
mat_file = tempfile("mat_data")
the file mat_data contains m, two rows, three columns
write(t(m), mat_file)
if(interactive()) file.show(mat_file)
unlink(mat_file)
Writing to the "console" 'tab-delimited'
two rows, three cols but the first row is 1 2 3 4 5 with a tab delimiter
write(m, "", sep = "\t")

The wrapper functions for write.table

There are numerous methods for exporting R objects into other formats;e.g., for Excel,we shall need the **xlsReadWrite package.**

- To export a Tab Delimited Text File, we write:
  write.table(Name-of-data-frame, "fileName.txt", sep="\t")
- To write to an Excel Spreadsheet:
  library(xlsx)
  write.xlsx(Name-of-data-frame, " fileName.xlsx")
- To create a .csv file in R, we write:
  write.csv(Name-of-data-frame, file="filename.csv")
- To create the file by omitting the rownames, we write:
  write.csv(Name-of-data-frame, file="filename.csv", row.names=FALSE)
- To omit the NA values, we modify the command as follows:
  write.csv(Name-of-data-frame, file="filename.csv", row.names=FALSE, na="")
- If we want to omit the column names also, we write the command as follows:
  write.table(Name-of-data-frame, file="filename.csv", row.names=FALSE, col.names= FALSE,na="", sep=",")

An illustrative example to show the processing of records of a file

Let us now develop an R script to create a student data file and then to read the records one by one , process each of the records to separate different components of the record and then to display them until the end of the file.
The following is the R script developed for the purpose.

```
{
# creating files of student records
cat("Enter the maximum number of records to be written")
m=scan(n=1)
for(i in 1:m){
cat("Record for student No.:",i)
print("comprising student name,course name and fees paid delimited by comma")
rec_data=readline("Enter your record here:=>")
write(rec_data,"student.dat",append=TRUE) # This is to write records one after another # append =TRUE is requitred to add on to the existing records
} # End of the loop for writing records
conn=file("student.dat","r") # conn sets up a connection to the file "student.txt"
while(TRUE){
rec=readLines(conn,n=1) # n=1 is required to read one record at a time
if(length(rec)==0){
print("End of file reached")
break
} n=nchar(rec)
pos=regexpr(",",rec) # This is to find out the position of the first comma in the record
sname=substr(rec,1,(pos-1)) # This is to separate the name from the record
rest1=substr(rec,pos+1,n) # This is to separate the rest part of the record
pos2=regexpr(",",rest1)  # This is to find out the comma separating the name of the Course
course=substr(rest1,1,pos2-1) #This is to separate the name of the course
m=nchar(rest1) # This is to find out the number of characters in the rest part of the record
rest2=substr(rest1,pos2+1,m) # The rest part of the record is separated now
pos3=regexpr(",",rest2)
Fees_paid=substr(rest2,pos3+1,m)
```

Continued · · · below.

Continuation of the preceding R script

```
cat("Name of the student is: ",sname)
cat("Course of the student is: ",course)
cat("Fees paid by the student is Rs.:",Fees_paid))
}
}
```

dput() and dget() Functions in R:

A more descriptive representation of an R object can be created by using the **dput()** or **dump()** functions. Unlike writing out a table or CSV file, **dump()** and **dput()** preserve the metadata, so that we need not specify it again during the next use. For example, we can preserve the class of each column of a table or the levels of a factor variable as illustrated below:

Let us create the following data frame

Let us now print the **'dput'** output to the R console

>**dput(x)**

Let us now write the **'dput'** output to a file

>**dput(x, file = "H://dputs.R")**

Now, to read in output of **dput()** from the file, we write:

>**result= dget("H:/dputs.R")**

>**result**

The dump() Function in R:

The **dump() function** is used to dump R objects to a file by passing its names as illustrated below.

>**hobby=readline("Enter your hobby=¿ ")**

>**df2= data.frame(Name = "Dr. ABC", Gender = "Male", Age=64)**

>**dump(c("hobby", "df2"), file = "H://dumped_data.R")**

>**rm(x, d)** #After dumping the data, we just remove the variables from the environment.

The source() Function in R:

The **source() function** is inverse of **dump()**. Now to import that dumped$_{d}ata$

>**source("H://dumped_data.R")**

Now, to print the values stored in the variables, we write:

>**hobby**

[1] "Gardening"

>**df2**

	Name	Gender	Age
1	Dr. ABC	Male	64

$\cdots$ Continued . . .

Continued Source() and Binary Formats

```
>str(df2)
'data.frame': 1 obs. of 3 variables:
$ Name : chr "Dr. ABC"
 Gender: chr "Male"
$ Age : num 64
```

Binary Formats in R:

Binary format is the the complement to the textual format. It is sometimes found to be useful to achieve efficiency. Sometimes, if there may not be any useful way to represent some data in a textual manner then binary format helps to import and export data in R. The main functions for converting R objects into a binary format are **save(), save.image(), and serialize()**.The **save() function** can be used to save individual R objects to a file. Let us have some illustrative use as shown below.

```
>dff=data.frame(NAME=c("Ramen","John"),AGE=c(23,25))
>var=runif(10,min=10,max=30)
>save(dff,var,file="H:/filedata.rdata")
>load("H:/filedata.rdata")
```

Saving image and serialize() and unserialize() function

As discussed earlier,to save everything to a file,we write:

>**save.image(file = "H://allmydata.RData")**

>**load("H://allmydata.RData")**

If we have a lot of objects that we want to save to a file in one run,we can save all objects in our workspace using the **save.image()** function.

The serialize() and unserialize() function in R:

The **serialize() function** is used to convert individual R objects into a binary format that can be communicated across an arbitrary connection. When we call the **serialize()** on an R object, **the output will be a raw vector coded in hexadecimal format**. The benefit of the **serialize() function** is that **it is the only way to perfectly represent an R object in an exportable format, without losing precision or any metadata.** If that is what we need, then **serialize()** is the function for us. We illustrate the use of the functions in the following R code:

>**lst=list(45,59,67)**

>**sl=serialize(lst,NULL)**

Serialization

```
>sl
[1] 58 0a 00 00 00 03 00 04 01 01 00 03 05 00 00 00 00 06 43 50 31 32
35 32 00
[26] 00 00 13 00 00 00 03 00 00 00 0e 00 00 00 01 40 46 80 00 00 00 00
00 00 00
[51] 00 0e 00 00 00 01 40 4d 80 00 00 00 00 00 00 00 00 0e 00 00 00 01
40 50 c0
[76] 00 00 00 00 00
>unserialize(sl)
[[1]]
[1] 45
[[2]]
[1] 59
[[3]]
[1] 67
```

saveRDS() and readRDS() in R

We have observed that **save()** allows us to save a named R object to a file or other connection and **load()**restores that object again to the current environment with the same name with which it had when saved. This creates a problem when we have a saved model object resulting from a previous fit and we want to compare it with the model object returned when the R code is rerun. Unless we change the name of the model fit object in our script we cannot have both the saved object and the newly created one available in the same environment at the same time. **saveRDS()** provides a solution to this problem and to the general one of saving and loading objects created with R. **saveRDS()** serializes an R object into a format that can be saved. It does not save the both the object and its name;it just saves a representation of the object. As a result, the saved object can be loaded into a named object within R that is different from the name it had when originally serialized. The main difference is that **save()** can save many objects to a file in a single call, whereas, **saveRDS()**, being a lower-level function, works with a single object at a time.The **readRDS()** function is used to read the file saved with **saveRDS()**.

saveRDS() and readRDS() in R–Contd.

The use of the functions has been illustrated in the following R code.
saving a single object to file
Let us consider the built-in women dataset comprising data on height and weight.
>**attach(women)**
We now save the data in women as shown below:
>**saveRDS(women,"H://Ladies1.rds")**
Let us now restore the saved data with the under a different name, Ladies2 as under:
>**Ladies2=readRDS("H://Ladies1.rds")**
If we need to compare any two similar sets of data, say,**women** and **Ladies2**,we write:
>**identical(women,Ladies2)**
[1] TRUE